Experiment 100 Human-Auditable Report

Square-Zero Nonunital Cofibrant Computation Through Weight 10

Jordan Quillen Homology Project

Run finished: 2026-06-03T04:47:17
Report updated with EXP101 generator-legality audit: 2026-06-03

Audit Scope

This report summarizes the actual run of `EXP-100-square-zero-nonunital-cofibrant-weight10`. The target is the nonunital Jordan algebra

$$J = \mathrm{Jord}\langle x\rangle/(x^2)$$

over $\mathbb{Q}$. The computation is truncated at product weight $w \leq 10$ and homological degree ≤ 2. It computes the bounded cell layers V_0, V_1, V_2, does not apply the functor Q, and does not construct any layers $V_3, V_4, \ldots$.

The value of the degree-2 layer was not assumed before the run. In particular, the plan and runner treated $\widetilde{J}_2$ as unknown data to be discovered by exact linear algebra. The setup file `expected.json` records only the experiment bounds and explicitly marks computed cells, differentials, predicted raw cycle counts, predicted minimal cell tables, and outcomes as fields not to prefill.

Primary Audit Artifacts

The full machine-readable audit trail is kept in JSON. This TeX file is a human-readable report and intentionally does not duplicate every basis vector or coordinate array.

```
Plan:
researchplan/subplan100.md

Runner:
experiments/100-square-zero-nonunital-cofibrant-weight10/run.py

Setup-only expectation file:
experiments/100-square-zero-nonunital-cofibrant-weight10/expected.json

Primary result object:
experiments/100-square-zero-nonunital-cofibrant-weight10/results.json

Per-weight audit data:
experiments/100-square-zero-nonunital-cofibrant-weight10/data/by_weight_W10.json

Cell audit data:
experiments/100-square-zero-nonunital-cofibrant-weight10/data/cells_W10.json

Run log:
```

```
experiments/100-square-zero-nonunital-cofibrant-weight10/logs/run_W10.log

Post-run generator-legality audit:
experiments/101-one-generator-generator-legality-audit/results.json
```

The reproducing command recorded for the run is:

```
python experiments/100-square-zero-nonunital-cofibrant-weight10/run.py --max-weight 10
```

Raw target field from `results.json`:

```
Jord<x>/(x^2)
```

Backend and Conventions

The run used exact rational arithmetic. The backend convention recorded in `results.json` is:

```
ordinary commutative nonassociative Jordan algebra modulo the linearized Jordan identity, with a
    signed derivation differential
```

The matrix convention is `source_rows_target_coordinates`. The output is therefore relative to this ordinary backend convention. A separate theory check is still needed before identifying this computation with a fully graded operadic dg Jordan convention.

Computation Procedure

The computation proceeds weight by weight.

1. Build V_0 from the presentation generator x, mapped to $\bar{x}$ in J.
2. For each weight $w \leq 10$, compute the presentation kernel in the ordinary nonunital Jordan backend. A new V_1 relation cell is accepted only when its differential maps to zero in J and its representative is independent modulo the prior closure generated by previously accepted relations.
3. After the accepted V_1-layer is known up to the current weight, form the old degree-one complex
$$C_{2,w}^{old} \xrightarrow{d_2^{old}} C_{1,w} \xrightarrow{d_1} C_{0,w}.$$
4. Compute $Z_{1,w} = \ker d_1$, $B_{1,w}^{old} = \mathrm{im}(d_2^{old})$, and $H_{1,w}^{old} = Z_{1,w}/B_{1,w}^{old}$. Candidate V_2 representatives are selected from certified cycles whose RREF remainder modulo old boundaries is nonzero and independent from previously selected representatives.
5. Record chain, rank-nullity, cycle, non-boundary, and independence checks in `results.json` and `data/by_weight_W10.json`.

Run Summary

Field	Value
Status	`run`
Passed	`true`
Requested maximum weight	10
Completed weights	$1, 2, 3, 4, 5, 6, 7, 8, 9, 10$
Skipped weights	none
Failed weights	none
Runtime recorded in `results.json`	0.116 seconds
Total V_0 cells	1
Total V_1 cells	3
Total V_2 cells	5

V_1 Discovery Audit

The following table records the relation-discovery pass. The column $\dim A_{0,w}$ is the source free-Jordan basis dimension in weight w; $\dim\ker \epsilon_w$ is the presentation-kernel dimension in that weight; "closure rank" is the rank of the relation closure after accepting any new minimal V_1 cells at that weight.

Raw V_1 cell names from `data/cells_W10.json`:

```
r1_00001_w2
r1_00002_w10
r1_00003_w10
```

Weight	$\dim A_{0,w}$	$\dim\ker \epsilon_w$	Closure rank	New V_1 cells
1	1	0	0	none
2	1	1	1	`r1_00001_w2`
3	1	1	1	none
4	1	1	1	none
5	1	1	1	none
6	2	2	2	none
7	2	2	2	none
8	5	5	5	none
9	8	8	8	none
10	17	17	17	`r1_00002_w10`, `r1_00003_w10`

Why new V_1 cells appear at weight 10. The two weight-10 relation cells are not an imposed prediction. In weight 10, the presentation kernel has dimension 17. Before adding the two weight-10 relation cells, the closure generated by earlier V_1 relations has rank 15. Thus two independent kernel directions remain outside the old closure. The runner chooses two representatives with nonzero RREF remainders, recorded below as sparse coordinates $\{0 : 1\}$ and $\{8 : 1\}$, and only then the closure rank reaches 17.

Accepted V_1 Cells

`r1_00001_w2`, **weight 2.** Differential:

```
(x*x)
```

Certificate summary: maps to 0 in J; RREF remainder modulo prior closure is nonzero; accepted as minimal in weight 2.

`r1_00002_w10`, **weight 10.** Differential:

```
((((((x*x)*x)*x)*x)*x)*((x*x)*(x*x)))
```

Certificate summary: maps to 0 in J; prior closure rank is 15; RREF remainder modulo prior closure is nonzero with sparse coordinate $\{0 : 1\}$; accepted as minimal in weight 10.

`r1_00003_w10`, **weight 10.** Differential:

```
(((((x*x)*x)*x)*x)*(((x*x)*x)*(x*x)))
```

Certificate summary: maps to 0 in J; prior closure rank is 16; RREF remainder modulo prior closure is nonzero with sparse coordinate $\{8 : 1\}$; accepted as minimal in weight 10.

Weight-by-Weight Homology Audit

For each completed weight, the runner records $C_{0,w}$, $C_{1,w}$, the old $C_{2,w}^{old}$, the ranks of d_1 and d_2^{old}, and $\dim H_{1,w}^{old}$. The check `dim_H1_old_matches_V2_count` is true in every row below.

w	$\dim C_0$	$\dim C_1$	$\dim C_2^{old}$	$\operatorname{rank} d_1$	$\dim Z_1$	$\operatorname{rank} d_2^{old}$	$\dim H_1^{old}$
1	1	0	0	0	0	0	0
2	1	1	0	1	0	0	0
3	1	1	0	1	0	0	0
4	1	2	1	1	1	0	1
5	1	2	2	1	1	1	0
6	2	4	4	2	2	2	0
7	2	7	7	2	5	4	1
8	5	14	17	5	9	8	1
9	8	29	37	8	21	21	0
10	17	65	90	17	48	46	2

w	New V_1	New V_2
1	none	none
2	`r1_00001_w2`	none
3	none	none
4	none	`s2_00001_w4`
5	none	none
6	none	none
7	none	`s2_00002_w7`
8	none	`s2_00003_w8`
9	none	none
10	`r1_00002_w10`, `r1_00003_w10`	`s2_00004_w10`, `s2_00005_w10`

Candidate V_2 Representatives Selected by the Old-Homology Audit

Each listed V_2 candidate below has three recorded EXP100 certificates: the chosen differential is a cycle ($d_1 z = 0$); its remainder modulo old boundaries is nonzero; and the selected representative is independent modulo old boundaries plus previously selected V_2 representatives. These certificates are old-complex certificates only; they are not, by themselves, strict dg attaching-legality certificates.

Raw V_2 cell names from `data/cells_W10.json`:

```
s2_00001_w4
s2_00002_w7
s2_00003_w8
s2_00004_w10
s2_00005_w10
```

`s2_00001_w4`, weight 4. Differential:

```
-((r1_00001_w2*x)*x) + ((x*x)*r1_00001_w2)
```

Cycle normal form: `0`. Boundary remainder modulo B^{old}:

```
-((r1_00001_w2*x)*x) + ((x*x)*r1_00001_w2)
```

Certificate type: `rref_remainder`; remainder nonzero; independent modulo B^{old} plus prior selected representatives.

`s2_00002_w7`, weight 7. Differential:

```
-(((((x*x)*x)*x)*x)*r1_00001_w2) + ((((x*x)*x)*x)*(r1_00001_w2*x))
```

Cycle normal form: `0`. Boundary remainder modulo B^{old}:

```
-1/2*(((r1_00001_w2*x)*(x*x))*(x*x)) + 1/2*(((x*x)*r1_00001_w2)*((x*x)*x))
```

Certificate type: `rref_remainder`; remainder nonzero; independent modulo B^{old} plus prior selected representatives.

`s2_00003_w8`, weight 8. Differential:

```
-((((((x*x)*x)*x)*x)*x)*r1_00001_w2) + 1/2*(((((x*x)*(x*x))*x)*x)*r1_00001_w2) - 1/2*(((((x*x)*x)*x
    )*r1_00001_w2)*(x*x)) + (((((x*x)*x)*x)*x)*(r1_00001_w2*x))
```

Cycle normal form: `0`. Boundary remainder modulo B^{old}:

```
1/4*((((x*x)*x)*((x*x)*x))*r1_00001_w2) - 1/4*((((x*x)*x)*(r1_00001_w2*x))*(x*x))
```

Certificate type: `rref_remainder`; remainder nonzero; independent modulo B^{old} plus prior selected representatives.

`s2_00004_w10`, weight 10. Differential:

```
2*(((((((x*x)*x)*x)*x)*x)*x)*x)*r1_00001_w2) - (((((((x*x)*x)*x)*x)*x)*r1_00001_w2)*(x*x)) -
    2*(((((((x*x)*x)*x)*x)*x)*x)*(r1_00001_w2*x)) + r1_00002_w10
```

Cycle normal form: `0`. Boundary remainder modulo B^{old}:

```
-(((((r1_00001_w2*x)*x)*x)*x)*((x*x)*(x*x))) + r1_00002_w10
```

Certificate type: `rref_remainder`; remainder nonzero; independent modulo B^{old} plus prior selected representatives.

s2_00005_w10, weight 10. Differential:

```
(((((((x*x)*x)*x)*x)*x)*x)*x)*r1_00001_w2) + 1/12*(((((((x*x)*(x*x))*x)*x)*x)*x)*r1_00001_w2) -
    1/12*(((((((x*x)*x)*(x*x))*x)*x)*x)*r1_00001_w2) + 1/4*(((((((x*x)*x)*x)*x)*x)*x)*r1_00001_w2)*(x*
    x)) - 1/3*(((((((x*x)*x)*x)*x)*x)*x)*(r1_00001_w2*x)) - 1/3*((((((x*x)*(x*x))*x)*x)*x)*x)*(
    r1_00001_w2*x)) - 11/12*((((((x*x)*x)*x)*x)*(x*x))*(r1_00001_w2*x)) - 2/3*((((((x*x)*x)*x)*x)*x
    )*((r1_00001_w2*x)*x)) + r1_00003_w10
```

Cycle normal form: `0`. Boundary remainder modulo B^{old}:

```
-((((r1_00001_w2*x)*x)*x)*(((x*x)*x)*(x*x))) + r1_00003_w10
```

Certificate type: `rref_remainder`; remainder nonzero; independent modulo B^{old} plus prior selected representatives.

Global Checks Recorded by the Runner

Check	Result
`all_V2_differentials_are_cycles`	true
`all_V2_representatives_have_nonzero_B_old_remainder`	true
`all_V2_representatives_independent_mod_B_old`	true
`all_completed_weights_passed`	true
`all_relation_cells_map_to_kernel`	true
`applies_Q_is_false`	true
`chain_condition_checks_passed`	true
`constructs_higher_cells_V3_plus_is_false`	true
`exact_rational_arithmetic`	true
`expected_setup_has_no_prefilled_results`	true
`rank_nullity_checks_passed`	true
`report_language_has_no_forbidden_global_phrasing`	true

Post-Run Generator-Legality Audit (EXP101)

EXP101 was run after EXP100 to test whether the generators selected in EXP100 are legal as strict dg attaching data in the same bounded backend. The audit uses EXP100's `results.json` and `data/cells_W10.json` as inputs, extends the legality check through weight 12, does not apply Q, does not construct $V_3, V_4, \ldots$, and makes no homology-killing claim.

EXP101 check	Result
Requested generator-legality bound	12
Reference EXP100 run passed	`true`
Declared generators satisfy $d^2 = 0$	`true`
V_1 relation generators legal through weight 12	3/3
V_2 generator-level differentials are cycles	5/5
V_2 candidates strictly attachable through weight 12	0/5
Strict attachability failures recorded	5

The first strict-attachability defects recorded by EXP101 are:

Candidate	First defect weight	Multiplier
`s2_00001_w4`	6	`(x*x)`
`s2_00002_w7`	8	`x`
`s2_00003_w8`	9	`x`
`s2_00004_w10`	11	`x`
`s2_00005_w10`	11	`x`

Thus EXP101 confirms the EXP100 V_1 relation generators through the tested range, but it does not validate the five chosen V_2 representatives as strict attaching differentials. A V_2 obstruction here means that the chosen representative fails the bounded multiplicative strictness test in $A^{(1)}$. It does not rule out a different representative, a corrected lift, or behavior outside the tested bound.

Result Interpretation

Within the recorded truncation $w \leq 10$, the run found:

$$|V_0| = 1, \qquad |V_1| = 3, \qquad |V_2| = 5.$$

The five V_2 cells occur in weights $4, 7, 8, 10, 10$. The two weight-10 V_2 cells explicitly involve the two new weight-10 V_1 relation cells `r1_00002_w10` and `r1_00003_w10`.

After EXP101, the safe interpretation is sharper: the three V_1 relation generators are legal through the tested weight 12, while the five EXP100 V_2 records should be read as old-complex cycle representatives rather than as validated strict V_2 attaching cells. This is bounded, backend-relative computational evidence. It does not prove global stabilization, does not assert absence of cells above weight 10, does not construct V_3 or higher layers, and does not identify the ordinary backend output with a fully graded operadic dg Jordan model without further theory work.